

Contents

1	SprintQuest / MSA Phase 2 Context Pack	2
1.1	0. Current project status - updated 28 July 2026	2
1.1.1	1.1.1 Production links	2
1.1.2	1.1.2 Current technical state	2
1.1.3	1.1.3 What currently works in the frontend	2
1.1.4	1.1.4 Known frontend gaps	3
1.1.5	1.1.5 Current development phase	3
1.2	1. Who I am and how I want to learn	3
1.2.1	1.2.1 Teaching style I want	3
1.2.2	1.2.2 My current skill context	3
1.3	2. Assessment context	4
1.3.1	1.3.1 Required theme	4
1.3.2	1.3.2 Basic assessment requirements	4
1.3.3	1.3.3 Advanced requirements	5
1.3.4	1.3.4 Required repository content	5
1.4	3. Project concept	6
1.5	Working title: SprintQuest	6
1.5.1	1.5.1 Core idea	6
1.5.2	1.5.2 Example board columns	6
1.5.3	1.5.3 Example task card fields	6
1.5.4	1.5.4 Example achievements	7
1.5.5	1.5.5 How the project fits the gamification theme	7
1.6	4. Recommended architecture	7
1.6.1	1.6.1 Backend layer responsibilities	7
1.6.2	1.6.2 Frontend structure	8
1.7	5. Git workflow	9
1.7.1	1.7.1 Main branches	9
1.7.2	1.7.2 Supporting branches	9
1.7.3	1.7.3 Normal workflow	10
1.7.4	1.7.4 Typical commands	10
1.7.5	1.7.5 Commit message style	10
1.8	6. GitHub Projects workflow	11
1.8.1	1.8.1 Project board name	11
1.8.2	1.8.2 Board statuses	11
1.8.3	1.8.3 Suggested fields	11
1.8.4	1.8.4 GitHub milestones	11
1.8.5	1.8.5 User story template	11
1.9	7. Milestone plan	12
1.9.1	1.9.1 M0 - Planning & Repository Setup	12
1.9.2	1.9.2 M1 - Backend Solution Setup	13
1.9.3	1.9.3 M2 - Core Domain Models	13
1.9.4	1.9.4 M3 - EF Core Database	14
1.9.5	1.9.5 M4 - CRUD API & Scalar Docs	14
1.9.6	1.9.6 M5 - Frontend Setup	14
1.9.7	1.9.7 M6 - Sprint Board UI	15
1.9.8	1.9.8 M7 - Gamification System	15
1.9.9	1.9.9 M8 - Zustand State Management	16
1.9.10	1.9.10 M9 - Security Measures	16
1.9.11	1.9.11 M10 - SignalR Live Board	16
1.9.12	1.9.12 M11 - Testing	17
1.9.13	1.9.13 M12 - Deployment	17
1.9.14	1.9.14 M13 - App Selection Foundation	18
1.9.15	1.9.15 M14 - Dashboard Integration	18
1.9.16	1.9.16 M15 - Projects & Sprints Management	19
1.9.17	1.9.17 M16 - Task Details & Checklist UI	20

1.9.18 M17 - Progress & Gamification UI	20
1.9.19 M18 - UX Polish, Responsive Design & Frontend Testing	21
1.9.20 M19 - README, Specs & Video	21
1.9.21 M20 - Final Release	22
1.10 8. Standard milestone starter prompt	22
1.11 9. Standard milestone closeout checklist	23
1.12 10. Standard end-of-milestone handoff prompt	23
1.13 11. Scope control rules	24
1.14 12. Current recommended next step	24

1 SprintQuest / MSA Phase 2 Context Pack

Use this file at the start of every new ChatGPT thread for this project.

Upload the Markdown or PDF version, or paste the relevant sections, so the assistant has the current project status, assessment requirements, architecture, Git workflow, learning preferences, and milestone plan.

1.1 0. Current project status - updated 28 July 2026

Milestones **M0 through M12 are complete**. SprintQuest is deployed and the production branch is main.

1.1.1 Production links

Frontend: <https://calm-forest-08e02e000.7.azurestaticapps.net>

Backend: <https://sprintquest-api-jd-2026-bpd5gyhzghhxgdgt.australiaeast-01.azurewebsites.net>

Scalar path: /scalar/v1 (append to the Backend URL)

1.1.2 Current technical state

- Backend: .NET 10, ASP.NET Core, EF Core, SQLite, Clean Architecture-style layers.
- Frontend: React, TypeScript, Vite, React Router, Zustand.
- Advanced features: Zustand state management, security validation/rate limiting, SignalR WebSockets.
- Production database: /home/sprintquest.db on Azure App Service.
- Production deployments: GitHub Actions deploy relevant changes from main.
- Backend tests: **85 passing**.
- Frontend tests: **15 passing across 3 test files**.
- SignalR live board updates work between browser clients.
- Database data survives App Service restarts.
- Duplicate task-completion XP is prevented with TaskItem.XpAwardedAt.
- Task story points must be between 1 and 100.

1.1.3 What currently works in the frontend

The **Sprint Board** is the only fully connected product page. It can:

- load tasks,
- create tasks,
- change task status,
- delete tasks,
- show sprint progress,
- show XP and achievements,
- receive live SignalR updates.

1.1.4 Known frontend gaps

The original frontend setup milestone created routes and placeholder pages, but later milestones focused on the board. These areas still need to be completed:

- Dashboard data integration,
- Projects CRUD interface,
- Sprint management interface,
- dynamic project and sprint selection,
- Task Details page,
- checklist management UI,
- Progress and achievements page,
- responsive/mobile navigation,
- loading, empty, error, and not-found states,
- frontend tests for all completed pages.

1.1.5 Current development phase

The next phase is **Application Completion**, covered by milestones M13-M18. Submission polish and final release move to M19-M20.

1.2 1. Who I am and how I want to learn

My name is **James Dunlop**. I am building this as part of the **Microsoft Student Accelerator 2026 Phase 2 Software Stream** assessment.

I want this project to be a learning project as well as a submission project.

1.2.1 Teaching style I want

Please follow these rules when helping me:

- Do **not** give me the whole project at once.
- Work in **small milestones** and **small coding steps**.
- Explain **why** before **what**.
- After each coding step, explain:
 - what changed,
 - why we did it,
 - where it fits architecturally,
 - how to test it,
 - when to commit it.
- Teach me like I am preparing for a real junior developer role.
- Use Git workflow guidance throughout, not only at the end.
- Use GitHub Projects, milestones, issues, user stories, and pull requests.
- Help me understand the code, not just copy it.
- If there is an error, debug it step by step with me.
- Keep the scope controlled so I can actually finish and submit.

1.2.2 My current skill context

I have already been learning and building projects using:

- C#
- ASP.NET Core Web API
- Clean Architecture-style project structure
- Entity Framework Core
- SQLite

- React + Vite
- SignalR
- REST APIs
- DTOs
- Controllers
- Dependency Injection
- xUnit tests
- Git and GitHub
- GitHub Issues and project planning
- Swagger/OpenAPI-style API testing

I have worked on projects such as:

- Barber Booking System
- Real-Time Chat app using SignalR
- Text RPG with C# domain logic, API, React UI, and tests

So for this project, please build on those concepts and connect new ideas back to things I already know.

1.3 2. Assessment context

This project is for **MSA 2026 Phase 2 | Software Stream**.

1.3.1 Required theme

The required theme is **Gamification**.

Gamification means adding game design elements to a non-game application. Examples include:

- points,
- XP,
- badges,
- achievements,
- streaks,
- leaderboards,
- progress tracking,
- levels,
- rewards.

The project does **not** have to be a game. It can be a useful app with game-like motivation features.

1.3.2 Basic assessment requirements

The application must include both a frontend and a backend.

1.3.2.1 Frontend requirements

- Built with **React**.
- **TypeScript is preferred**.
- Visually appealing and responsive UI.
- Navigation using React Router or similar.
- Clear Git usage with regular commit history.
- Deployed frontend.
- Frontend unit tests covering key components/functionality.

1.3.2.2 Backend requirements

- Built using **C# with .NET 10 or higher**.
- Must use **Entity Framework Core**.
- Must use database persistence with SQL or NoSQL.
- Must implement CRUD operations.
- Clear Git usage with regular commit history.
- Backend unit tests covering key backend components/functionality.
- Deployed backend.
- Must expose **Scalar API documentation UI instead of Swagger UI**.

1.3.3 Advanced requirements

At least **three advanced requirements** must be implemented and explicitly listed in the README. Only the top three advanced features listed in the README will be marked.

Chosen top three advanced features for this project:

1. **State management library**: Zustand
2. **Security measures**: at least two, such as password hashing, data validation/sanitisation, rate limiting, or RBAC
3. **WebSockets**: SignalR live board updates

Optional extra if time allows:

- Theme switching with light/dark mode
- Docker
- Cypress end-to-end tests
- Caching/API optimisation

1.3.4 Required repository content

The repo should contain:

- frontend code,
- backend code,
- README,
- /specs folder,
- deployment links,
- video link for submission.

The README should include:

- deployment link,
- project introduction,
- how the project relates to gamification,
- interesting/unique features,
- clear checklist of top 3 advanced features,
- self-reflection,
- setup instructions,
- testing instructions.

The /specs folder should contain .md files with evidence of:

- planning,
- design,
- AI-assisted development,
- AI prompts used during development,
- agent instructions,
- context/config files.

The submission video must be **maximum 6 minutes** and should include:

- how AI was used during development,
 - design decisions made during the project.
-

1.4 3. Project concept

1.5 Working title: SprintQuest

SprintQuest is a **gamified project development canvas board**.

It is similar to a small Trello/Jira-style board, but designed for students or developers managing software projects through sprints, tasks, checklists, and milestones.

The app helps users plan project work and stay motivated by awarding XP, achievements, streaks, and sprint progress.

1.5.1 Core idea

Users can create:

- projects,
- sprints,
- task cards,
- checklist items,
- statuses/columns,
- priorities,
- story points,
- progress ticks.

Gamification features include:

- XP rewards when tasks are completed,
- XP rewards when checklist items are ticked,
- sprint progress bars,
- achievements/badges,
- project level,
- streaks,
- “sprint complete” bonus.

1.5.2 Example board columns

- Backlog
- To Do
- In Progress
- Testing
- Done

1.5.3 Example task card fields

- title,
- description,
- status,
- priority,
- story points,
- XP reward,
- due date,
- checklist progress,
- created date,
- completed date.

1.5.4 Example achievements

- **First Task Complete:** Complete your first task.
- **Sprint Finisher:** Complete all tasks in a sprint.
- **Bug Slayer:** Complete 5 bug tasks.
- **Momentum Builder:** Complete tasks 3 days in a row.
- **Clean Board:** Move all sprint tasks to Done.
- **Deep Work:** Complete a high story-point task.

1.5.5 How the project fits the gamification theme

SprintQuest applies gamification to project planning. Instead of being a game, it uses game-like elements such as XP, achievements, progress bars, streaks, and completion rewards to make project development feel more motivating and engaging.

1.6 4. Recommended architecture

Use a Clean Architecture-style backend similar to my previous projects.

```
SprintQuest/
├── backend/
│   ├── SprintQuest.Api
│   ├── SprintQuest.Application
│   ├── SprintQuest.Domain
│   ├── SprintQuest.Infrastructure
│   └── SprintQuest.Tests
├── frontend/
│   └── sprintquest-ui
├── specs/
│   ├── 01-project-plan.md
│   ├── 02-ai-prompts.md
│   ├── 03-architecture.md
│   ├── 04-design-decisions.md
│   ├── 05-advanced-features.md
│   └── 06-testing-plan.md
├── README.md
└── .gitignore
```

1.6.1 Backend layer responsibilities

1.6.1.1 SprintQuest.Domain Contains the core business models and rules.

Examples:

- Project
- Sprint
- TaskItem
- ChecklistItem
- Achievement
- XpEvent
- TaskStatus enum
- Priority enum

Business rules that belong here:

- completing a task sets CompletedAt,
- completing a task awards XP,
- checklist progress affects task progress,
- sprint completion can trigger bonus XP,
- achievement logic can be tested separately.

The Domain layer should not depend on EF Core, controllers, or React.

1.6.1.2 SprintQuest.Application Contains application logic, DTOs, interfaces, and use cases.

Examples:

- DTOs for create/update/read operations,
- service interfaces,
- application services,
- validation rules,
- request/response models.

This layer coordinates actions but should not directly know about the database implementation.

1.6.1.3 SprintQuest.Infrastructure Contains database and external implementation details.

Examples:

- EF Core DbContext,
- repositories,
- database migrations,
- entity configuration,
- seed data.

Infrastructure depends on Domain/Application, not the other way around.

1.6.1.4 SprintQuest.Api Contains the ASP.NET Core Web API.

Examples:

- Controllers,
- SignalR hubs,
- auth setup,
- Scalar API documentation,
- dependency injection wiring,
- middleware,
- API endpoints.

Controllers should stay thin. They should call services rather than containing all business logic.

1.6.1.5 SprintQuest.Tests Contains unit tests for:

- domain rules,
- application services,
- key backend behavior.

1.6.2 Frontend structure

Use React + TypeScript + Vite.

Recommended structure:

frontend/sprintquest-ui/

```
|
|— src/
|   |— api/
|   |— components/
|   |— features/
|   |   |— projects/
|   |   |— sprints/
|   |   |— board/
|   |   |— tasks/
|   |   |— gamification/
|   |   |— auth/
|   |— pages/
|   |— routes/
|   |— stores/
|   |— types/
|   |— utils/
|   |— tests/
```

Use Zustand for shared frontend state such as:

- selected project,
 - selected sprint,
 - board state,
 - task filters,
 - theme state,
 - progress summary.
-

1.7 5. Git workflow

Use Git flow style.

1.7.1 Main branches

main
develop

- main is only for stable, release-ready versions.
- develop is the integration branch where completed feature branches are merged.

1.7.2 Supporting branches

feature/*
bugfix/*
hotfix/*
release/*
docs/*
test/*

Use examples like:

feature/backend-solution-setup
feature/domain-models
feature/efcore-database
feature/crud-api-scalar
feature/frontend-setup
feature/sprint-board-ui
feature/gamification-system

feature/zustand-state-management
feature/security-measures
feature/signalr-live-board

bugfix/task-status-update
hotfix/deployment-env-error
release/v1.0.0-msa-submission
docs/readme-skeleton
docs/specs-folder
test/backend-domain-tests

1.7.3 Normal workflow

```
main
↓
develop
↓
feature/something
↓
Pull Request into develop
↓
develop
↓
release/v1.0.0
↓
main
```

1.7.4 Typical commands

Start from develop:

```
git checkout develop
git pull
git checkout -b feature/example-name
```

After work:

```
git status
git add .
git commit -m "feat: add example feature"
git push -u origin feature/example-name
```

Then open a Pull Request:

feature/example-name → develop

Use this in PR descriptions to close issues:

Closes #ISSUE_NUMBER

1.7.5 Commit message style

Use simple conventional commits:

```
feat: add project entity
fix: correct task status update
docs: add architecture notes
test: add task completion tests
chore: create backend solution
refactor: move XP logic into domain
```

1.8 6. GitHub Projects workflow

Use GitHub Projects to track the work.

1.8.1 Project board name

SprintQuest MSA Phase 2

1.8.2 Board statuses

Backlog
Ready
In Progress
Review
Done

1.8.3 Suggested fields

Priority: High / Medium / Low
Type: Feature / Bug / Docs / Test / Chore
Area: Backend / Frontend / Domain / API / Infrastructure / Specs
Milestone: M0, M1, M2...
Story Points: 1, 2, 3, 5, 8

1.8.4 GitHub milestones

M0 - Planning & Repository Setup	COMPLETE
M1 - Backend Solution Setup	COMPLETE
M2 - Core Domain Models	COMPLETE
M3 - EF Core Database	COMPLETE
M4 - CRUD API & Scalar Docs	COMPLETE
M5 - Frontend Setup	COMPLETE
M6 - Sprint Board UI	COMPLETE
M7 - Gamification System	COMPLETE
M8 - Zustand State Management	COMPLETE
M9 - Security Measures	COMPLETE
M10 - SignalR Live Board	COMPLETE
M11 - Testing	COMPLETE
M12 - Deployment	COMPLETE
M13 - App Selection Foundation	NEXT
M14 - Dashboard Integration	
M15 - Projects & Sprints Management	
M16 - Task Details & Checklist UI	
M17 - Progress & Gamification UI	
M18 - UX Polish, Responsive Design & Frontend Testing	
M19 - README, Specs & Video	
M20 - Final Release	

1.8.5 User story template

Use this for GitHub issues:

User Story

As a *[type of user]*,

I want [some feature],
so that [benefit/value].

Acceptance Criteria

- [] Criteria 1
- [] Criteria 2
- [] Criteria 3

Suggested Subtasks

- [] Subtask 1
- [] Subtask 2
- [] Subtask 3

Notes

Any design decisions, AI prompts, or important context.

1.9 7. Milestone plan

1.9.1 M0 - Planning & Repository Setup

Goal: define the project before coding.

Create:

- README skeleton,
- /specs folder,
- project plan,
- architecture plan,
- AI prompts file,
- GitHub Project board,
- GitHub milestones,
- first issues/user stories,
- main and develop branches.

Suggested branches:

docs/project-plan
docs/architecture-plan
docs/readme-skeleton
docs/specs-folder
docs/git-flow-setup

Definition of Done:

- repo exists,
 - README skeleton exists,
 - /specs folder exists,
 - GitHub Project exists,
 - M0 issues exist,
 - develop branch exists,
 - first PR merged into develop.
-

1.9.2 M1 - Backend Solution Setup

Goal: create the .NET backend solution structure.

Create:

- SprintQuest.Api
- SprintQuest.Application
- SprintQuest.Domain
- SprintQuest.Infrastructure
- SprintQuest.Tests

Learn:

- project references,
- why Domain is separate,
- how API connects to Application/Infrastructure,
- where tests belong.

Branch:

feature/backend-solution-setup

Definition of Done:

- solution builds,
 - references are correct,
 - first basic test runs,
 - branch merged into develop.
-

1.9.3 M2 - Core Domain Models

Goal: create the core business models.

Create:

- Project,
- Sprint,
- TaskItem,
- ChecklistItem,
- Achievement,
- XpEvent,
- TaskStatus enum,
- Priority enum.

Learn:

- entities vs DTOs,
- business rules,
- domain methods,
- unit testing domain behavior.

Branch:

feature/domain-models

Definition of Done:

- domain models exist,
 - basic rules implemented,
 - tests for key domain behavior pass,
 - branch merged into develop.
-

1.9.4 M3 - EF Core Database

Goal: persist data using EF Core.

Create:

- DbContext,
- DbSet,
- EF Core packages,
- database migration,
- SQLite/PostgreSQL setup,
- entity relationships.

Branch:

feature/efcore-database

Definition of Done:

- migration works,
 - database created,
 - data persists,
 - branch merged into develop.
-

1.9.5 M4 - CRUD API & Scalar Docs

Goal: expose backend CRUD APIs.

Create:

- DTOs,
- controllers,
- services,
- validation,
- Scalar API docs.

Endpoints should cover:

- Projects CRUD,
- Sprints CRUD,
- Tasks CRUD,
- Checklist Items CRUD.

Branch:

feature/crud-api-scalar

Definition of Done:

- CRUD works,
 - Scalar API docs work,
 - manual endpoint testing complete,
 - branch merged into develop.
-

1.9.6 M5 - Frontend Setup

Goal: create the React TypeScript frontend.

Create:

- Vite React TypeScript app,
- React Router,

- page structure,
- API service structure,
- basic responsive layout.

Pages:

- Dashboard,
- Project page,
- Sprint board page,
- Task details page,
- Profile/progress page.

Branch:

feature/frontend-setup

Definition of Done:

- frontend runs,
 - routing works,
 - basic pages exist,
 - branch merged into develop.
-

1.9.7 M6 - Sprint Board UI

Goal: build the canvas board.

Create:

- board columns,
- task cards,
- status updates,
- create/update/delete tasks from frontend,
- API integration.

Branch:

feature/sprint-board-ui

Definition of Done:

- board displays tasks,
 - task status can change,
 - frontend calls backend,
 - branch merged into develop.
-

1.9.8 M7 - Gamification System

Goal: strongly connect the project to the gamification theme.

Create:

- XP rewards,
- achievement unlocks,
- sprint progress,
- project/user level,
- gamification UI.

Branch:

feature/gamification-system

Definition of Done:

- completing tasks awards XP,
 - achievements unlock,
 - progress is visible,
 - tests cover key gamification rules,
 - branch merged into develop.
-

1.9.9 M8 - Zustand State Management

Goal: implement advanced feature 1.

Use Zustand for:

- selected project,
- selected sprint,
- board state,
- filters,
- progress summary.

Branch:

feature/zustand-state-management

Definition of Done:

- Zustand installed,
 - stores created,
 - state moved out of messy component state,
 - README advanced feature checklist updated,
 - branch merged into develop.
-

1.9.10 M9 - Security Measures

Goal: implement advanced feature 2.

Implement at least two:

- password hashing,
- data validation/sanitisation,
- rate limiting,
- RBAC/authorisation.

Branch:

feature/security-measures

Definition of Done:

- at least two security measures implemented,
 - README explains why they matter,
 - tests added where appropriate,
 - branch merged into develop.
-

1.9.11 M10 - SignalR Live Board

Goal: implement advanced feature 3.

Create:

- BoardHub,
- live task moved event,
- live task completed event,
- live achievement unlocked event,
- frontend SignalR connection.

Branch:

feature/signalr-live-board

Definition of Done:

- SignalR hub works,
 - frontend receives board updates,
 - README explains WebSockets feature,
 - branch merged into develop.
-

1.9.12 M11 - Testing

Goal: satisfy frontend and backend testing requirements.

Backend:

- xUnit,
- domain tests,
- service tests.

Frontend:

- Vitest,
- React Testing Library,
- key component tests.

Branch:

test/unit-tests

Definition of Done:

- backend tests pass,
 - frontend tests pass,
 - README has test instructions,
 - branch merged into develop.
-

1.9.13 M12 - Deployment

Goal: deploy frontend and backend.

Deploy:

- frontend,
- backend,
- database if needed.

Update:

- README deployment links,
- environment variable notes,
- CORS config if needed.

Branch:

release/deployment-prep

Definition of Done:

- deployed frontend works,
 - deployed backend works,
 - frontend can call backend,
 - deployment links are in README,
 - branch merged into develop.
-

1.9.14 M13 - App Selection Foundation

Goal: create the shared frontend foundation needed by every unfinished page and remove hard-coded production assumptions.

Why this milestone comes first:

- Dashboard, Projects, Task Details, Progress, and the Board must agree on the selected project and sprint.
- The board currently depends on a configured default sprint ID.
- Shared selection state prevents each page from inventing its own state and API-loading logic.

Create:

- an audit of DashboardPage, ProjectsPage, TaskDetailsPage, ProgressPage, routes, navigation, API clients, and stores,
- frontend Sprint and checklist types where missing,
- a sprintsApi client and any missing project/checklist API methods,
- shared Zustand state for selected project and selected sprint,
- initial project/sprint loading,
- persistence of the selected IDs where appropriate,
- clear handling when there are no projects or no sprints,
- a route/navigation decision for Task Details,
- a safe transition away from the hard-coded/default production sprint ID.

Important design rule:

The application should use the user's selected sprint. VITE_DEFAULT_SPRINT_ID may remain as a temporary fallback during the transition, but it should not be the final source of truth.

Suggested branch:

feature/app-selection-foundation

Definition of Done:

- project and sprint selection rules are documented,
 - selected project and sprint state are available through Zustand,
 - missing frontend API/type foundations exist,
 - the Board can use shared sprint selection or a clearly documented temporary fallback,
 - loading, empty, and API-error behaviour is defined,
 - focused frontend tests pass,
 - branch merged into develop.
-

1.9.15 M14 - Dashboard Integration

Goal: replace the Dashboard placeholder cards with real production data.

Create:

- active project count,
- selected/current sprint summary,
- completed and remaining task counts,

- sprint completion percentage,
- total XP,
- unlocked achievement summary,
- links or actions that take the user to Projects, Board, or Progress,
- loading, empty, and error states,
- responsive dashboard layout.

Do not add new backend analytics unless the existing endpoints cannot support a required value. Prefer composing the current Projects, Sprints, Tasks, and Gamification APIs first.

Suggested branch:

feature/dashboard-integration

Definition of Done:

- no placeholder milestone text remains on the Dashboard,
 - Dashboard cards display real API data,
 - empty databases show useful onboarding actions,
 - API errors are visible and recoverable,
 - Dashboard component tests pass,
 - branch merged into develop.
-

1.9.16 M15 - Projects & Sprints Management

Goal: make the Projects page a complete management area and let users choose the work context used by the Board.

Create:

- project list,
- create project form,
- edit project flow,
- delete project confirmation,
- selected project state,
- sprint list for the selected project,
- create sprint form,
- edit sprint flow,
- delete sprint confirmation,
- selected sprint state,
- navigation from a sprint into the Board,
- validation messages from the API,
- loading, empty, and error states.

Scope control:

- no drag-and-drop project hierarchy,
- no team permissions,
- no complex scheduling calendar,
- use the existing CRUD API before adding backend features.

Suggested branch:

feature/projects-sprints-ui

Definition of Done:

- users can complete project CRUD from the frontend,
- users can complete sprint CRUD from the frontend,
- selecting a sprint updates shared state,
- the Board loads the selected sprint,
- destructive actions require confirmation,

- Projects page tests pass,
 - branch merged into develop.
-

1.9.17 M16 - Task Details & Checklist UI

Goal: make `/tasks/:taskId` a functional details and editing page.

Create:

- navigation from a task card to its details route,
- task loading by route ID,
- editable title and description,
- editable status, priority, story points, and XP reward,
- created/completed/XP-awarded information where appropriate,
- checklist item list,
- create checklist item,
- update checklist item text/completion state,
- delete checklist item,
- loading, not-found, validation, and API-error states,
- live reconciliation where SignalR updates affect the open task.

Navigation rule:

Remove the static **Task Details** sidebar link. Task Details should be opened from a real task card because the route requires a task ID.

Suggested branch:

`feature/task-details-checklists`

Definition of Done:

- clicking a task opens `/tasks/:taskId`,
 - users can edit and save task details,
 - checklist CRUD works from the frontend,
 - an invalid task ID shows a useful not-found state,
 - Task Details tests pass,
 - branch merged into develop.
-

1.9.18 M17 - Progress & Gamification UI

Goal: turn the Progress placeholder into a focused gamification and sprint-performance page.

Create:

- selected sprint completion percentage,
- completed versus remaining tasks,
- task status distribution,
- total XP and XP-event count,
- unlocked achievement cards,
- clear empty states for a new sprint,
- links back to the Board and Projects,
- responsive progress visuals.

Scope control:

Use current gamification data first. New levels, streaks, leaderboards, or charts are optional only after the required page works and tests pass.

Suggested branch:

feature/progress-gamification-ui

Definition of Done:

- no placeholder content remains on Progress,
 - progress values match the selected sprint and gamification summary,
 - achievements are displayed clearly,
 - loading, empty, and error states work,
 - Progress page tests pass,
 - branch merged into develop.
-

1.9.19 M18 - UX Polish, Responsive Design & Frontend Testing

Goal: make the completed application feel consistent, responsive, and submission-ready.

Create or improve:

- responsive sidebar or mobile navigation,
- active navigation states,
- consistent page headers and card styling,
- keyboard-accessible controls and labels,
- confirmation and success feedback,
- application-level 404 route,
- consistent loading, empty, validation, and API-error components,
- tests for Dashboard, Projects, Task Details, Progress, navigation, and shared stores,
- production smoke testing across frontend, REST API, SignalR, and persistence.

Optional after required work:

- theme switching,
- a small end-to-end smoke test,
- visual refinements that do not expand feature scope.

Suggested branch:

feature/frontend-completion-polish

Definition of Done:

- all navigation destinations are functional,
 - mobile and desktop layouts are usable,
 - placeholder milestone content is removed,
 - frontend test coverage includes every main page,
 - backend and frontend test suites pass,
 - the deployed application is smoke-tested,
 - branch merged into develop and released to main.
-

1.9.20 M19 - README, Specs & Video

Goal: prepare submission materials after the application pages are complete.

Finish:

- README,
- deployment links,
- screenshots of all functioning pages,
- /specs evidence,
- AI prompt evidence,
- design decision notes,
- top-three advanced feature checklist,

- self-reflection,
- setup and testing instructions,
- maximum 6-minute video script and recording plan.

Suggested branch:

docs/submission-polish

Definition of Done:

- README accurately describes the finished application,
 - screenshots match the deployed version,
 - specs are complete,
 - video script is timed to six minutes or less,
 - branch merged into develop.
-

1.9.21 M20 - Final Release

Goal: perform the final release and submission check.

Suggested branch:

release/v1.0.0-msa-submission

Checklist:

- Dashboard works,
- Projects and Sprints management works,
- Sprint Board works,
- Task Details and checklists work,
- Progress and achievements work,
- responsive navigation works,
- frontend deployed,
- backend deployed,
- database working,
- README complete,
- /specs complete,
- video public,
- GitHub repo public,
- no secrets committed,
- all tests pass,
- no placeholder milestone content remains,
- no commits after the submission deadline.

Final merge path:

develop -> release/v1.0.0-msa-submission -> main

1.10 8. Standard milestone starter prompt

Use this at the start of each new ChatGPT thread.

I am working on my MSA 2026 Phase 2 Software Stream project called SprintQuest.

I have uploaded my SprintQuest Context Pack. Please read it and continue using that context.

Current milestone:

[WRITE MILESTONE NUMBER AND NAME HERE]

Current Git branch:
[WRITE BRANCH NAME HERE]

Current goal:
[WRITE WHAT I AM TRYING TO COMPLETE HERE]

Important teaching rules:

- Do not give me the whole project at once.
 - Work step by step.
 - Explain why before what.
 - Include Git workflow guidance.
 - Help me create GitHub user stories/issues for the milestone.
 - After each coding step, explain what changed and why it matters.
 - Explain where it fits architecturally.
 - Explain how to test it and when to commit.
 - At the end of the milestone, create a handoff prompt for the next thread.
-

1.11 9. Standard milestone closeout checklist

At the end of every milestone, check:

Milestone Closeout

- [] All user stories are completed
 - [] Acceptance criteria are checked
 - [] Tests pass
 - [] App still runs
 - [] README/specs updated if needed
 - [] Branch committed
 - [] Branch pushed
 - [] Pull request opened
 - [] Pull request merged into develop
 - [] GitHub issues closed
 - [] End-of-milestone prompt created
-

1.12 10. Standard end-of-milestone handoff prompt

Use this to start the next thread.

I have completed [MILESTONE NUMBER AND NAME] for SprintQuest.

Completed:

- [LIST WHAT WAS COMPLETED]
- [LIST TESTS OR CHECKS DONE]
- [LIST BRANCH MERGED INTO DEVELOP]

Current repo state:

- main branch is stable
- develop branch includes completed milestone work
- latest completed branch: [BRANCH NAME]

Please start [NEXT MILESTONE NUMBER AND NAME] with me.

Follow my project rules:

- Explain why before what
 - Work step by step
 - Create GitHub user stories/issues first
 - Include Git workflow guidance
 - Do not give me the whole project at once
 - Teach me the architecture as we go
-

1.13 11. Scope control rules

To keep the project finishable:

- Do not build a full Jira clone.
 - Finish Dashboard, Projects/Sprints, Task Details/Checklists, and Progress before adding optional features.
 - Do not add drag-and-drop unless every required page works and tests pass.
 - Do not add authentication, teams, RBAC, comments, attachments, notifications, or a calendar unless required for assessment.
 - Reuse the existing backend CRUD endpoints before creating new endpoints.
 - Keep project and sprint selection in shared state rather than duplicating it across pages.
 - Remove the static Task Details navigation link; open details from a selected task.
 - Prefer simple progress cards and bars before complex chart libraries.
 - Do not add Docker unless the required work is stable.
 - Update README/specs during each milestone rather than leaving everything to the last day.
 - Commit regularly.
 - Test small pieces as they are built.
 - Keep the top 3 advanced features clear in the README.
-

1.14 12. Current recommended next step

Start with:

M13 - App Selection Foundation

Why:

The Sprint Board works, but the unfinished pages need a shared project/sprint context. Building that foundation first prevents Dashboard, Projects, Task Details, Progress, and the Board from using conflicting or hard-coded state.

First tasks:

1. Confirm develop contains all completed M12 work and is clean.
2. Create the M13 GitHub milestone and issues.
3. Audit the four placeholder pages, routes, sidebar, existing API clients, and Zustand stores.
4. Document how selected project and selected sprint should flow through the app.
5. Create branch feature/app-selection-foundation from develop.
6. Add missing Sprint/checklist frontend types and API methods.
7. Add shared selected-project and selected-sprint state.
8. Add focused tests before connecting the remaining pages.

Suggested first-thread goal:

Audit the current frontend and design the selected project/sprint state flow.

Do not implement all pages at once.